

E-COMMERCE ANALYTICS PLATFORM

Advanced Application Development Project

Technical Report

FURKAN DEMİRCİ (20220808035)
BERKAY BARIŞ ÇALIŞKAN (20210808012)

[Click Here for GitHub Repository Link](#)

26 Nisan 2026

Contents

1	Introduction and Project Summary	2
2	Architecture Decisions and Rationale	2
2.1	Core Technology Stack and Selection Reasons	2
2.2	Frontend Routing and Request Pipeline (SPA Architecture)	3
2.3	Layered Security Architecture (Defense in Depth)	3
3	Database Design and ER Diagram	3
3.1	Core Table Structures and Relationships	4
4	Authentication Architecture and Payment Integration	8
4.1	JWT Authentication and Token Lifecycle	8
4.2	OAuth2 Social Login (Google & GitHub)	9
4.3	Stripe Payment Integration	9
5	ETL Process and Field Mappings	9
5.1	Source Datasets Used	9
5.2	Transformation Processes and Data Cleansing	10
5.3	Data Field Mapping Documentation	11
6	API Documentation Summary	11
7	AI Chatbot Architecture	11
7.1	Architecture and Framework	12
7.2	Agent Pipeline and Execution Flow	12
7.3	Security Architecture	13
7.4	Role-Based AI Access Control	13
8	Challenges Encountered and Applied Solutions	14
8.1	Privilege Escalation via Frontend-Supplied Store ID in AI Requests . .	14
8.2	LangGraph Error-Correction Loop Entering Infinite Execution	14
8.3	Missing <code>@Transactional</code> on a <code>@Modifying</code> Repository Query	14
9	Docker and Containerization Infrastructure	15
9.1	Service Architecture	15
9.2	Multi-Stage Dockerfiles	16
9.3	Networking and Reverse Proxy	16
9.4	Environment Variable Management	16
9.5	CI/CD Pipeline	17
10	Conclusion	17
A	API Documentation (Swagger / OpenAPI)	18

1 Introduction and Project Summary

This report details the technical architecture, database design, API specifications, and integrated artificial intelligence (AI) based chatbot system of a comprehensive, enterprise-grade e-commerce analytics platform. The project comprises a **Spring Boot 3.5.11** based REST API backend running on **Java 21**, a modular single-page application frontend powered by **Angular 19.2**, and a multi-agent AI Analytical Chatbot infrastructure built with **FastAPI + LangGraph** (Python 3.11) utilizing OpenAI’s **GPT-4o-mini** model for natural language understanding. The system’s data persistence layer is managed by **PostgreSQL 16**, while identity and access management combines stateless **JWT** (JJWT 0.12.6) authentication with **OAuth2** social login via Google and GitHub providers. Payment processing is handled through a secure **Stripe** integration (stripe-java 24.22.0) with full webhook support. The system’s integrity is reinforced by a layered security model encompassing role-based access control (INDIVIDUAL, CORPORATE, ADMIN), an AI-layer rate-limiting and enumeration-detection filter, and a LangGraph guardrails agent that intercepts unsafe or out-of-scope natural language queries before any SQL is generated or executed.

2 Architecture Decisions and Rationale

The project design follows a decoupled microservices approach. The frontend, backend, and AI engine are orchestrated as independent services communicating exclusively over REST.

2.1 Core Technology Stack and Selection Reasons

To maximize performance, scalability, and security, the following stack was selected:

Table 1: Project Technology Stack and Version Matrix

Component	Core Technology	Port	Key Dependencies
Backend	Spring Boot 3.5.11 (Java 21)	8080	Spring Security, Spring Data JPA, JJWT 0.12.6, stripe-java 24.22.0, Spring OAuth2 Client
Frontend	Angular 19.2.0	4200 / 80	TypeScript 5.7.2, RxJS 7.8.0, Bootstrap 5.3.8, Chart.js 4.5.1, jspdf 4.2.1, xlsx 0.18.5
AI Engine	Python 3.11 (FastAPI)	8000	LangGraph, LangChain, langchain-openai (GPT-4o-mini), pandas, psycopg2-binary
Database	PostgreSQL 16	5432	Relational Database Management System (RDBMS)

- **Backend (Spring Boot 3.5.11 — Java 21):** Builds RESTful APIs, enforces

security via Spring Security, and manages data through Spring Data JPA. Supports MySQL and PostgreSQL, ensuring a database-agnostic architecture.

- **Frontend (Angular 19.2.0):** Delivers a fast, modular SPA. RxJS handles async data flows. Chart.js provides visualizations, while jspdf and xlsx enable client-side exports.
- **AI Chatbot (FastAPI, LangGraph):** Isolated RESTful service for analytical queries. LangGraph provides deterministic inter-agent communication using OpenAI's `gpt-4o-mini`.

2.2 Frontend Routing and Request Pipeline (SPA Architecture)

The Angular 19 SPA client-side security perimeter relies on two guards and an interceptor:

- **auth.guard.ts:** Checks `AuthService.isAuthenticated()` before routing. Unauthenticated users are redirected to `/login`.
- **role.guard.ts:** Enforces RBAC by comparing `route.data['roles']` against the user's role. Unauthorized access routes to `/forbidden`.
- **auth.interceptor.ts:** Attaches the `Authorization: Bearer <token>` header to outbound requests. Handles 401 `Unauthorized` errors by triggering a silent token refresh, queuing concurrent requests via an RxJS `BehaviorSubject` to prevent duplicate calls.

2.3 Layered Security Architecture (Defense in Depth)

A three-layered defense model protects the Text-to-SQL pipeline against prompt injection and data exfiltration:

1. **Client Layer:** Navigation guards block unauthorized access. The interceptor ensures valid tokens, and a custom `sanitizeResponse()` method masks technical error traces with user-friendly messages.
2. **API Layer:** A `JwtAuthFilter` validates tokens statelessly. An `AiRateLimitFilter` enforces a sliding-window limit of **10 req/min** on `/api/ai/**` endpoints using `ConcurrentHashMap`. Method-level authorization is secured via `@PreAuthorize`.
3. **AI Service Layer:** The `guardrails_agent` is the first node for user queries. It blocks out-of-scope questions, simple greetings, and injection attempts, ensuring only valid analytical requests reach the SQL generation node.

3 Database Design and ER Diagram

The system is built on a normalized, modular database architecture managed by PostgreSQL 16. There are no explicit migration files (Flyway/Liquibase); the schema is auto-generated and maintained by Hibernate using `spring.jpa.hibernate.ddl-auto=update`. All entities extend a common `BaseEntity @MappedSuperclass` that contributes `created_at` and `updated_at` audit columns. The design strictly adheres to the Relational Database model and enforces referential integrity via JPA relationship annotations.

3.1 Core Table Structures and Relationships

The database comprises **18 tables** derived from JPA entity classes. The **5 most critical core tables** are listed below with their key roles in the system:

- **users**: Central identity table. Holds email, hashed password, OAuth2 provider (LOCAL / GOOGLE / GITHUB), and role (INDIVIDUAL / CORPORATE / ADMIN).
- **stores**: Seller accounts. Each store maps 1:1 to an owning user via `owner_id`; multi-store logic is not present.
- **products**: Catalog items linked to a store (`store_id`) and a category (`category_id`). Stores SKU, unit price, and stock quantity.
- **categories**: Hierarchical tree via a self-referential `parent_id` foreign key.
- **orders**: Supports a parent-child (Master-SubOrder) hierarchy through a self-referential `parent_order_id`. Carries status, grand total, and shipping cost.

Table 2: Core Database Entity-Relationship Summary

Source Entity	Target Entity	Relationship Type
Users	Stores	1:1
Users	Orders	1:N
Stores	Products	1:N
Products	Categories	N:1
Categories	Categories	N:1 (self-ref parent)
Orders	Orders	1:N (parent-child hierarchy)

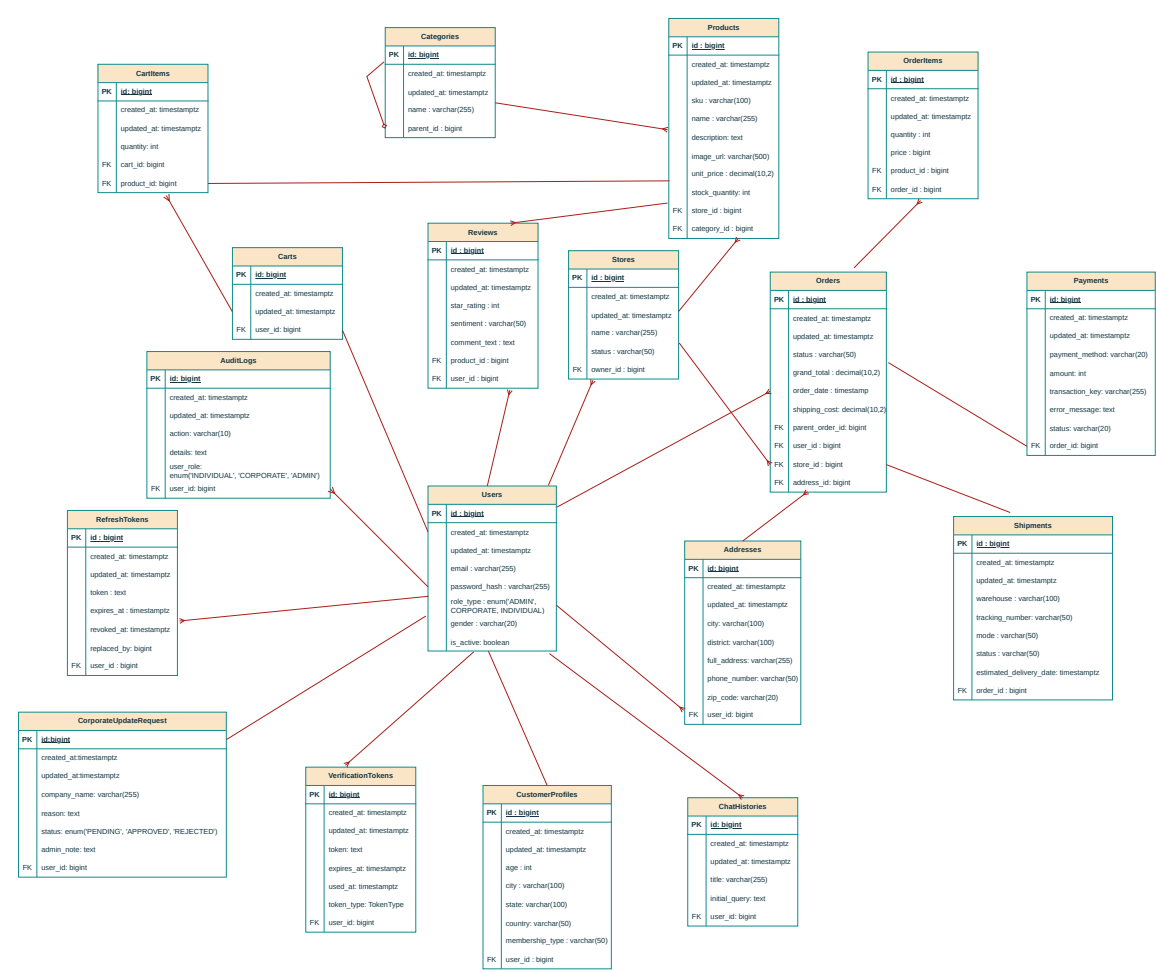


Figure 1: Physical Database ER (Entity-Relationship) Diagram of the System

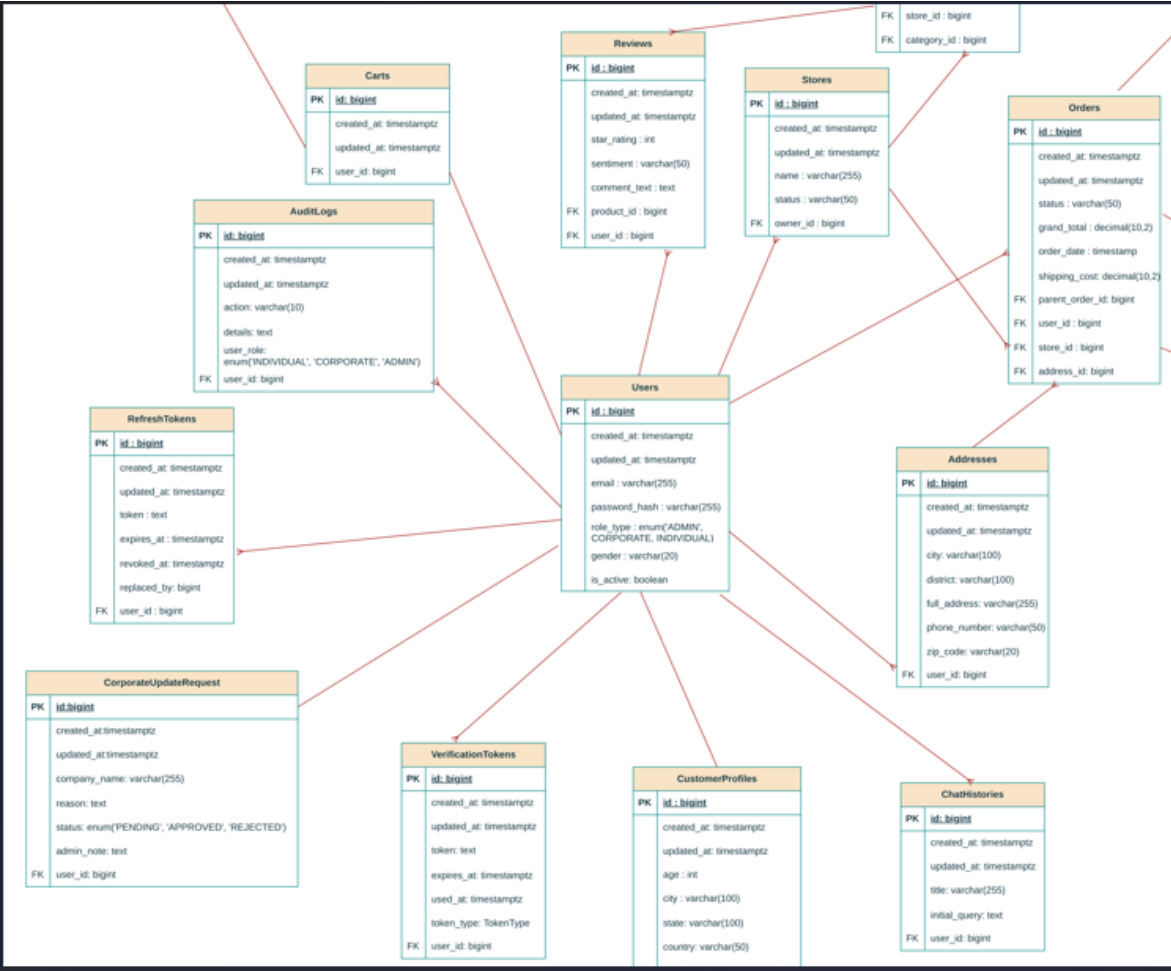


Figure 2: ER Diagram of the Users Cluster (including Identity, Profile, and Security modules)

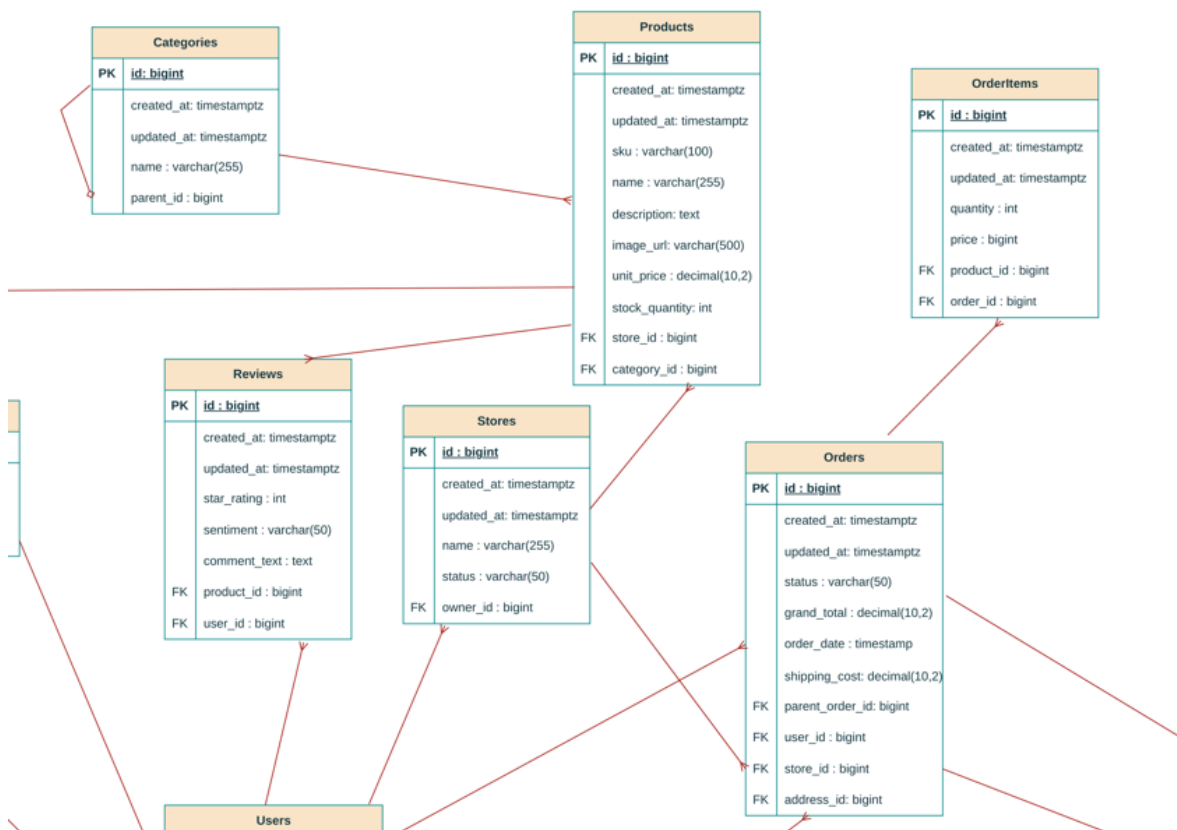


Figure 3: ER Diagram of the Products Cluster (including Catalog, Categories, and Store relationships)

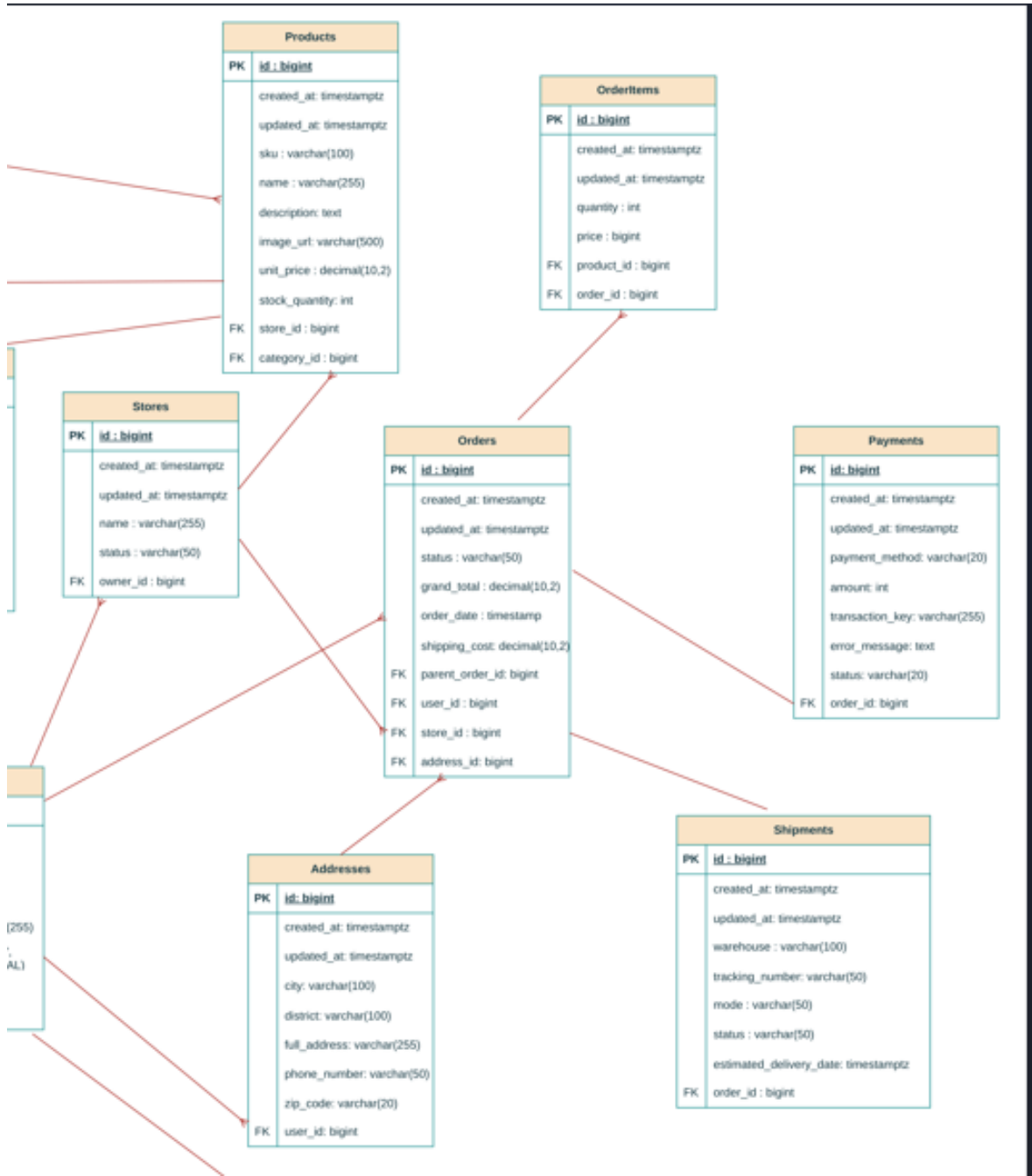


Figure 4: ER Diagram of the Orders Cluster (including Transactional, Payment, and Shipping modules)

4 Authentication Architecture and Payment Integration

This section details the multi-provider authentication pipeline (JWT + OAuth2) and Stripe payment integration, both implemented at the infrastructure level across all user flows.

4.1 JWT Authentication and Token Lifecycle

The system employs a stateless model using **JJWT 0.12.6** with two token types:

- **Access Token:** HS256 JWT (15-min expiry) containing the user's email and roles, transmitted via the `Authorization: Bearer` header.
- **Refresh Token:** UUID (7-day expiry) stored as an `HttpOnly, SameSite=Lax` cookie for XSS resistance, supporting auditable token rotation chains.

`JwtAuthFilter` validates tokens on every request. Angular's `auth.interceptor.ts` handles client-side silent refreshes, queuing concurrent requests to prevent duplicate calls.

4.2 OAuth2 Social Login (Google & GitHub)

One-click login is managed by `spring-boot-starter-oauth2-client`. The summarized flow:

1. Spring Security exchanges the provider's authorization code for an access token.
2. `CustomOAuth2UserService` extracts the email and upserts the `users` table. New users are auto-provisioned with `INDIVIDUAL` roles, profiles, and carts.
3. On success, old tokens are revoked, new ones are issued (cookie + JWT), and the user is redirected to the Angular callback. Failures safely route back to `/login`.

4.3 Stripe Payment Integration

`PaymentServiceImpl` manages orders based on three methods:

- **CREDIT_CARD:** Uses `stripe-java` to create a Checkout Session, stores the ID as `transactionKey`, and redirects the client to Stripe's hosted checkout.
- **BANK_TRANSFER & CASH_ON_DELIVERY:** Handled internally within the database.

Webhooks: A secure POST `/api/payments/webhook` endpoint processes Stripe callbacks using `STRIPE_WEBHOOK_SECRET`. It handles `checkout.session.completed` (updates status to `SUCCESS` and deducts inventory) and `expired` (updates to `FAILED`). Credentials are securely injected via Docker variables.

5 ETL Process and Field Mappings

Extract, Transform, Load (ETL) processes populated the platform with realistic test data. The pipeline uses real-world open-source CSV datasets transformed by Python scripts, alongside **Faker**-based synthetic seeders for structural data (stores, orders, shipments). Scripts connect directly to PostgreSQL via `psycopg2` and are additive (idempotent), safely allowing re-runs without data deletion. The execution order is strictly:

1. `seed.py` — Base stores, categories, synthetic users/products/orders.
2. `etl_csv.py` — Real Amazon products + Customer Behavior users.
3. `seed_extra.py` — 30 additional corporate stores.
4. `seed_reviews.py` — Star-rated review bank for new store products.
5. `seed_audit_logs.py` — Realistic admin/corporate audit entries.

5.1 Source Datasets Used

The following datasets were integrated into the architecture:

Table 3: Source Datasets Integrated into the Project

#	Dataset / Source	Key Fields	Target Tables
1	Amazon Sales (<code>amazon_sales.csv</code>)	<code>product_id</code> , <code>product_name</code> , <code>category</code> , <code>price</code> , <code>rating</code> , <code>review</code>	<code>products</code> , <code>categories</code> , <code>reviews</code>
2	E-commerce Customer Behavior	<code>Customer ID</code> , <code>Age</code> , <code>City</code> , <code>Membership</code> , <code>Satisfaction</code>	<code>users</code> , <code>profiles</code> , <code>carts</code> , <code>ad-</code> <code>dresses</code>
3	Pakistan Orders (<code>orders_etl.py</code>)	<code>increment_id</code> , <code>status</code> , <code>created_at</code> , <code>sku</code> , <code>qty</code> , <code>grand_total</code>	<code>orders</code> , <code>order_items</code> , <code>pay-</code> <code>ments</code>
4	Faker Synthetic Seed (<code>seed.py</code>)	50 users, 300 orders, 200 re- views, stores, shipments	<code>stores</code> , <code>products</code> , <code>orders</code> , <code>pay-</code> <code>ments</code>
5	Template Review Bank	Hardcoded English reviews (6 templates per star level)	<code>reviews</code>
6	Audit Log Seed	Admin & corporate action templates with parameter IDs	<code>audit_logs</code>

Note: Shipping and user segmentation datasets were intentionally skipped as they lack linkable identifiers to preserve relational integrity.

5.2 Transformation Processes and Data Cleansing

- **Currency Parsing:** Strips symbols (e.g. `₹`) and commas via regex, converting to `Decimal` clamped between 9.99–999,999.99.
- **Hierarchical Categories:** Pipe-delimited Amazon strings are split to upsert full parent-leaf paths into `categories`.
- **Rating Normalisation:** Rounded to 1–5 stars with a ± 1 variance to simulate natural distribution.
- **Review Text Extraction:** Snippets > 10 chars are separated, capped at 4 per product, and truncated at 1,000 chars.
- **Synthetic Emails:** Customer IDs generate deterministic emails (e.g., `cust.behavior.{cid}@example.com`) for users.
- **Type Mappings & Clamping:** string tiers map to `smallint` enums (Gold→2). Ages clamped to 18–90. Product names truncated to 240 chars.
- **Image Assignment:** Categories match a 17-rule `_CAT_IMAGE_RULES` keyword dictionary; failing that, defaults to `cat-general.jfif`.
- **Duplicate Prevention:** Enforced via `ON CONFLICT ...DO NOTHING` (SKU, Email) and in-memory sets during execution.

5.3 Data Field Mapping Documentation

Table 4: Comprehensive ETL Field Mapping Table

Source Field	Target DB Column	Applied ETL Rule & Decision Logic
product_name	products.name	Truncated to 240 chars; whitespace stripped.
product_id	products.sku	Prefix: AMZN-{id[:30]}; ON CONFLICT DO NOTHING.
price	products.unit_price	Strip non-numeric → Decimal; clamp 9.99–999,999.99; fallback 199.99.
category	categories.name, parent_id	Pipe-split → parent/leaf upsert via get_or_create_category().
rating	reviews.star_rating	Float → round → clamp 1–5; apply ±1 random variance.
review_content	reviews.comment_text	Comma-split → > 10 chars; max 4/product; 1,000 char limit.
Customer ID	users.email	Synth: cust.behavior.{cid}@demo.com; lowercase.
Age	customer_profiles.age	Int cast; clamped 18–90; fallback 30.
Membership	customer_profiles.membership_type	Gold→2, Silver→1, Bronze→0 enum map.
City	customer_profiles.city	Direct copy; truncated at 100 chars.
category (Amzn)	products.image_url	Keyword match via _CAT_IMAGE_RULES; fallback default.
qty_ordered	order_items.quantity	Positive values only; zero/negatives dropped.
grand_total	payments.amount	Cast to NUMERIC(18,2); nulls skipped.

6 API Documentation Summary

By its architectural nature, the Spring Boot RESTful backend serves 92 endpoints across 17 distinct controllers. The APIs are fully documented via Swagger/OpenAPI and secured by method-level `@PreAuthorize` annotations, enforcing strict role-based access control (`INDIVIDUAL`, `CORPORATE`, `ADMIN`).

The fundamental core endpoints, accurately reflecting the implemented micro-modules, are summarized below:

Table 5: Core REST API Service Map

Method	Endpoint	Process Description
POST	/api/auth/register	User Registration (Individual or Corporate)
POST	/api/auth/login	JWT Generation and User Authentication
GET	/api/products	Catalog: List All Products
GET	/api/products/store/{storeId}	CORPORATE: List Store-Specific Products
POST	/api/carts/add-item	Cart: Add Product to Current User’s Cart
POST	/api/orders/create	Order Processing: Generate Master and Sub-Orders
POST	/api/payments/create	Payments: Initiate Stripe Transaction
POST	/api/payments/webhook	Payments: Secure Stripe Webhook Listener
POST	/api/shipments/initialize	Shipments: Trigger Order Fulfillment Process
POST	/api/reviews	Catalog: Submit Product Review
GET	/api/analytics/store/{id}/*	CORPORATE: Retrieve Store Analytics & Customers
POST	/api/ai/ask/individual	AI Agent: Natural Language Queries (User Scope)
POST	/api/ai/ask/corporate	AI Agent: Natural Language Queries (Store Scope)
GET	/api/admin/audit-logs	ADMIN: View System-Wide Audit Logs

7 AI Chatbot Architecture

The platform integrates a multi-agent **Text-to-SQL** system allowing natural language database queries. Implemented in Python, it runs as an independent **FastAPI** microservice (port 8000). CORS is restricted to `localhost:8080` (Spring Boot backend).

Data access is role-scoped (ADMIN, CORPORATE, INDIVIDUAL) at both SQL generation and execution layers.

7.1 Architecture and Framework

Table 6: AI Agent Technology Stack

Component	Technology
Orchestration	LangGraph (stateful directed graph, <code>graph.py</code>)
LLM	OpenAI GPT-4o-mini via LangChain ChatOpenAI
API Layer	FastAPI + Uvicorn, POST /chat, GET /health
Database Driver	psycpg2-binary (PostgreSQL 16)
Safety Filter	NeMo Guardrails (<code>config.yml</code> + <code>self_check_input.yml</code>)
Visualization	Chart.js v3 config rendered via QuickChart.io
Shared State	AgentState TypedDict — 21 fields (e.g., <code>user_role</code> , <code>sql_query</code> , <code>error</code> , <code>history</code>)

The API accepts a ChatRequest (question, user_role, user_id, store_id) and returns final_answer, visualization_code, suggestions, and debugging traces. Prior conversation history is retained in AgentState for context.

7.2 Agent Pipeline and Execution Flow

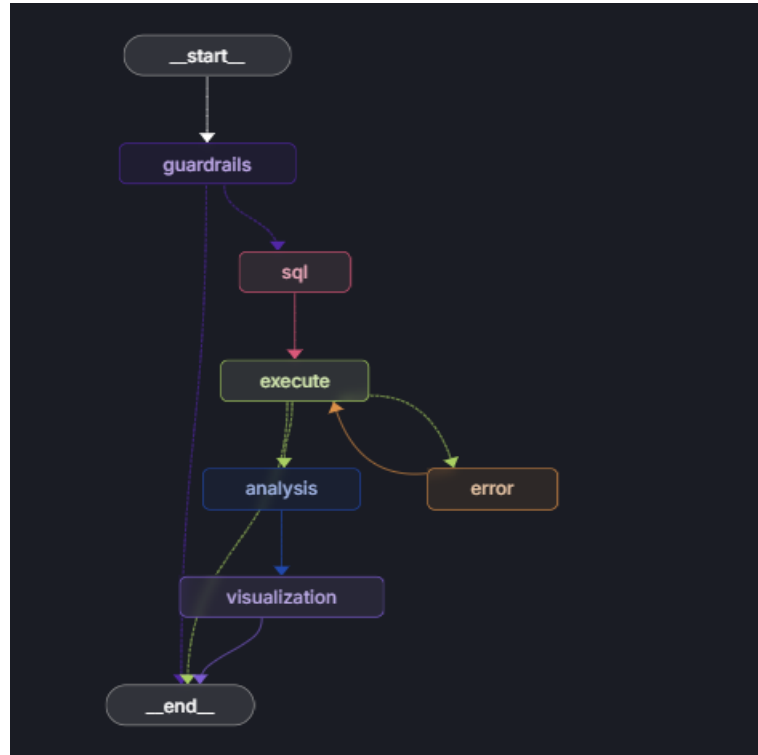


Figure 5: LangGraph AI Agent Execution Pipeline

The LangGraph pipeline uses six nodes. The nominal path is: `guardrails` → `sql` → `execute` → (error loop if needed) → `analysis` → `visualization` → END.

1. **guardrails_agent (Intent & Safety):** Classifies intent (`sql_query`, `greeting`, `off_topic`, `unsafe`). Unsafe queries terminate immediately. NeMo Guardrails performs secondary input checks.
2. **sql_agent (NL to SQL):** Generates `SELECT` queries via GPT-4o-mini using schema and history. Injects a mandatory role-scoped `WHERE` clause. Detects anaphoric references and emits `HIGH_TRAFFIC_WARNING` for 5+ sequential ID queries.
3. **db_executor (Secure Execution):** Regex blocks write keywords (`INSERT`, `UPDATE`, `DROP`, etc.). Strips 12 sensitive columns (`password_hash`, `api_key`, etc.) from results, capping at 50 rows.
4. **error_agent (Correction Loop):** Classifies DB errors and prompts LLM correction (max 3 iterations). Scope-widening (removing `WHERE` clauses) is explicitly forbidden.
5. **analysis_agent (Explanation):** Translates results into role-tailored summaries. Includes diagnostic sub-queries for empty results to prevent misleading outputs. Enforces sensitive data masking.
6. **visualization_agent (Charts):** Selects Chart.js types (line, doughnut, bar, radar) based on data shape. Excludes IDs/timestamps from Y-axis. Renders via QuickChart.io.

7.3 Security Architecture

Five independent layers ensure no single failure exposes data. Validated via `test_security_attacks`.

Table 7: AI Security Layers and Attack Vectors Mitigated

Layer	Component	Mitigated Threats
L1	guardrails + NeMo	Prompt injection, jailbreaks, SQL in NL, prompt leakage, cross-user theft.
L2	sql_agent scope	Horizontal privilege escalation: mandatory <code>WHERE</code> clause per role.
L3	db_executor list	Write/destructive query execution blocked; 12 sensitive columns stripped.
L4	api.py rate limit	Brute-force/enumeration: 10 req/min per user. 10-min block on sequential hits.
L5	analysis_agent	Indirect leakage: explanations never echo hashes, tokens, or keys.

7.4 Role-Based AI Access Control

The SQL agent injects role-specific constraints directly into the LLM system prompt before generation.

Table 8: Role-Based SQL Scope Injection

Role	Injected <code>WHERE</code> Constraint	Accessible Data
ADMIN	None	Full platform data.
CORPORATE	<code>WHERE store_id = {id}</code>	Own store’s products, orders, revenue, reviews.
INDIVIDUAL	<code>WHERE user_id = {id}</code>	Own orders, cart, reviews, addresses.

This constraint survives the entire pipeline. Natural-language bypass attempts are intercepted by L1, ignored by L2, and preserved during L4 error correction, ensuring strict boundary enforcement.

8 Challenges Encountered and Applied Solutions

As with any complex system, building this multi-module platform came with its share of technical hurdles. This section highlights three specific challenges drawn directly from the project’s commit history, breaking down the initial symptoms, the underlying root causes, and the solutions ultimately deployed.

8.1 Privilege Escalation via Frontend-Supplied Store ID in AI Requests

Problem. In early iterations the Angular chatbot component sent the current user’s `storeId` directly inside the `POST /api/ai/ask/corporate` request body. This meant any authenticated user could craft a request with an arbitrary `storeId` and ask the AI agent to query a different store’s orders, revenue, or product data — a textbook horizontal privilege-escalation attack (catalogued internally as AV-02).

Solution. The `storeId` and `userId` fields were removed from the request body entirely. On the Spring Boot side, `AIServiceImpl` was updated to extract both values directly from the authenticated principal via `SecurityContextHolder`, which is populated from the verified JWT by the stateless filter chain. The Angular frontend was updated to stop sending these fields. As a complementary control, `AiRateLimitFilter` was introduced in the same commit to enforce a sliding-window rate limit of ten requests per minute per user (AV-09). After this change, the scope boundary is enforced server-side and cannot be overridden by the caller under any circumstances.

8.2 LangGraph Error-Correction Loop Entering Infinite Execution

Problem. The LangGraph pipeline’s error-correction loop (`error_agent`) used an `iteration_count` counter to limit retries to three attempts. However, when the `sql_agent` or `error_agent` determined that a query was unrecoverable, it returned one of three sentinel strings — `SCOPE_VIOLATION`, `INJECTION_DETECTED`, or `UNFIXABLE` — as the value of `sql_query`. The `execute_sql_node` then passed this string directly to `psycpg2`, which raised a PostgreSQL syntax error. The `check_error` conditional edge routed back to `error_agent`, and because `iteration_count` was already at the maximum, the graph emitted no `final_answer` and returned `None` to the API caller instead of a user-readable message.

Solution. A `BLOCKED_SIGNALS` set was introduced in `graph.py`. The `execute_sql_node` inspects the `sql_query` value before touching the database; if it is a blocked signal, the node sets a canned error message in `final_answer` and returns without executing any SQL. The `check_error` and `should_continue` conditional edges were also updated to short-circuit to `END` whenever `final_answer` is already populated, preventing any further routing. A separate fallback was added for the genuine max-retry case: when `iteration_count` reaches three, a user-friendly bilingual message is written to `final_answer` before the graph exits.

8.3 Missing @Transactional on a @Modifying Repository Query

Problem. During OAuth2 social login the `OAuth2AuthenticationSuccessHandler` calls `RefreshTokenRepository.revokeAllByUser()` to invalidate any existing refresh tokens before issuing a new one. The method was annotated with Spring Data’s `@Modifying` and used a JPQL `UPDATE` statement, but was missing `@Transactional`.

Spring Data JPA requires that `@Modifying` methods execute within an active transaction; without it, a `TransactionRequiredException` was thrown at runtime. The failure only surfaced on the OAuth2 code path — standard username/password login was unaffected because its service layer was already wrapped in a transactional method — making the root cause difficult to locate initially.

Solution. Adding `@Transactional` directly on the repository method resolved the exception. The annotation ensures Spring opens a transaction for the `UPDATE` regardless of whether the caller’s service method is itself transactional. The fix was a single-line change, but the diagnostic effort highlighted a broader rule: every `@Modifying` JPQL method in a Spring Data repository must carry its own `@Transactional` annotation and cannot rely on the outer service transaction being present in all call paths.

9 Docker and Containerization Infrastructure

The entire platform is containerized with Docker and orchestrated by a single `docker-compose.yml` at the repository root. All four services start with one command:

```
docker compose up --build
```

A `.env` file in the project root supplies every secret and configuration value; no credentials are hard-coded in any image or compose definition.

9.1 Service Architecture

The compose file defines **four services** that together form the complete running system.

Table 9: Docker Compose Services

Service	Base Image	Port Mapping	Role
postgres	postgres:16-alpine	5432:5432	PostgreSQL 16 database; data persisted to named volume <code>postgres_data</code>
backend	eclipse-temurin:21-jre (built)	8080:8080	Spring Boot REST API; depends on <code>postgres</code> and <code>ai-agent</code>
ai-agent	python:3.11-slim (built)	<i>internal only</i> (<code>expose: 8000</code>)	FastAPI + LangGraph Text-to-SQL microservice; reachable only from <code>backend</code>
frontend	nginx:alpine (built)	80:80	Angular SPA served by Nginx; depends on <code>backend</code>

A notable security decision is that `ai-agent` uses `expose` rather than `ports`. This means port 8000 is only reachable by other containers on the same Docker network and is **never published to the host**. External clients have no direct path to the AI microservice; all traffic must pass through the authenticated Spring Boot API.

All services carry `restart: unless-stopped` to recover automatically from crashes without manual intervention.

9.2 Multi-Stage Dockerfiles

Each service that requires a build step uses a **multi-stage Dockerfile** to keep the final image as small as possible.

Backend (backend/ecommerce-backend/Dockerfile):

1. *Build stage* — `maven:3.9-eclipse-temurin-21`: copies `pom.xml`, pre-fetches dependencies offline, then compiles and packages the JAR (`-DskipTests`).
2. *Runtime stage* — `eclipse-temurin:21-jre`: copies only the final `app.jar`; the Maven toolchain and source tree are discarded.

Frontend (frontend/e-commerce-frontend/Dockerfile):

1. *Build stage* — `node:20-alpine`: runs `npm ci` for reproducible installs, then builds the Angular application in production mode.
2. *Runtime stage* — `nginx:alpine`: copies the compiled `dist/` output and a custom `nginx.conf`.

AI Agent (Text to SQL Agent/Dockerfile): Single-stage build on `python:3.11-slim`; installs dependencies from `requirements.txt` with `-no-cache-dir`, then starts Uvicorn bound to `0.0.0.0:8000`.

9.3 Networking and Reverse Proxy

Docker Compose creates a **default bridge network** automatically; no explicit `networks` block is required. All services are addressed by their service names:

- `backend` connects to the database at `postgres:5432`.
- `backend` forwards AI requests to `http://ai-agent:8000/chat` (set via the `AI_AGENT_URL` environment variable).
- The Nginx container reverse-proxies all `/api/*` requests to `http://backend:8080/api/`, so the Angular SPA makes no cross-origin requests from the browser's perspective.
- All other routes in Nginx fall through to `index.html`, enabling Angular's client-side router to handle deep links.

9.4 Environment Variable Management

All secrets are injected at runtime via a `.env` file placed in the repository root. Docker Compose interpolates `${VAR}` references in `docker-compose.yml` from this file. No secret is baked into any image layer.

Table 10: Environment Variables Required in `.env`

Variable	Consumer	Purpose
DB_USERNAME / DB_PASS	postgres, backend, ai-agent	Database credentials shared across services
JWT_SECRET	backend	HS256 signing key for access tokens
OPENAI_API_KEY	backend, ai-agent	OpenAI API access for GPT-4o-mini
STRIPE_SECRET_KEY / STRIPE_WEBHOOK_SECRET_KEY	backend	Stripe payment processing and webhook verification
GOOGLE_CLIENT_ID / GOOGLE_CLIENT_SECRET	backend	Google OAuth2 social login
GITHUB_CLIENT_ID / GITHUB_CLIENT_SECRET	backend	GitHub OAuth2 social login
MAIL_HOST / MAIL_PORT / MAILTRAP_USERNAME / MAILTRAP_PASSWORD	backend	SMTP credentials for transactional email (Mailtrap)

9.5 CI/CD Pipeline

A GitHub Actions workflow (`.github/workflows/ci-cd.yml`) automates image builds and pushes on every commit to `main` and on every pull request targeting `main`.

The pipeline runs **three parallel jobs** — one per buildable service (`backend`, `ai-agent`, `frontend`) — each following the same pattern:

1. Authenticate to **GitHub Container Registry** (`ghcr.io`) using the built-in `GITHUB_TOKEN` secret.
2. Build the Docker image using `docker/build-push-action@v5`.
3. Push the tagged image (`ghcr.io/<owner>/ecommerce-{service}:latest`) only on `push` events; pull-request builds validate the image but do not publish.

The `postgres` service is not included in the CI pipeline because it uses the official upstream image directly and requires no custom build.

10 Conclusion

This project demonstrates the design and implementation of a production-grade, full-stack e-commerce analytics platform built from three tightly integrated modules: a Spring Boot 3.5.11 / Java 21 REST API exposing 92 endpoints across 17 controllers, an Angular 19.2 single-page application with role-gated routing and a concurrent JWT refresh queue, and a Python FastAPI microservice orchestrating a six-node LangGraph Text-to-SQL pipeline powered by GPT-4o-mini. The platform enforces a three-layer security model — Angular functional guards and interceptor, Spring Security stateless filter chain with `AiRateLimitFilter`, and LangGraph guardrails with five independent defense layers covering ten catalogued attack vectors — ensuring that role boundaries hold under adversarial conditions as verified by an automated security test suite. Persistent data is managed by PostgreSQL 16 across 18 normalized tables; the schema is populated by a five-step ETL pipeline that integrates real-world open-source datasets with Faker-generated synthetic records, all via idempotent `ON CONFLICT` upserts. Payment processing is handled end-to-end through Stripe Checkout Sessions

and webhook-driven status reconciliation, while social login is supported via Google and GitHub OAuth2 with transactional token-rotation semantics. The entire system is containerized with multi-stage Docker builds and orchestrated through a single `docker compose up --build` command, with a GitHub Actions CI/CD pipeline automatically building and publishing all three service images to the GitHub Container Registry on every merge to `main`. Taken together, the project applies modern software engineering practices — layered security, global pagination, event-driven webhooks, multi-agent AI pipelines, and infrastructure-as-code — in a coherent, deployable system, providing a solid foundation for future extensions such as real-time notifications, multi-warehouse inventory management, or horizontal scaling via container orchestration.

A API Documentation (Swagger / OpenAPI)

The full interactive API documentation is available at <http://localhost:8080/swagger-ui/index> when the backend is running. All 92 endpoints across 17 controllers are listed with their request/response schemas, required roles, and example payloads generated by Springdoc OpenAPI.

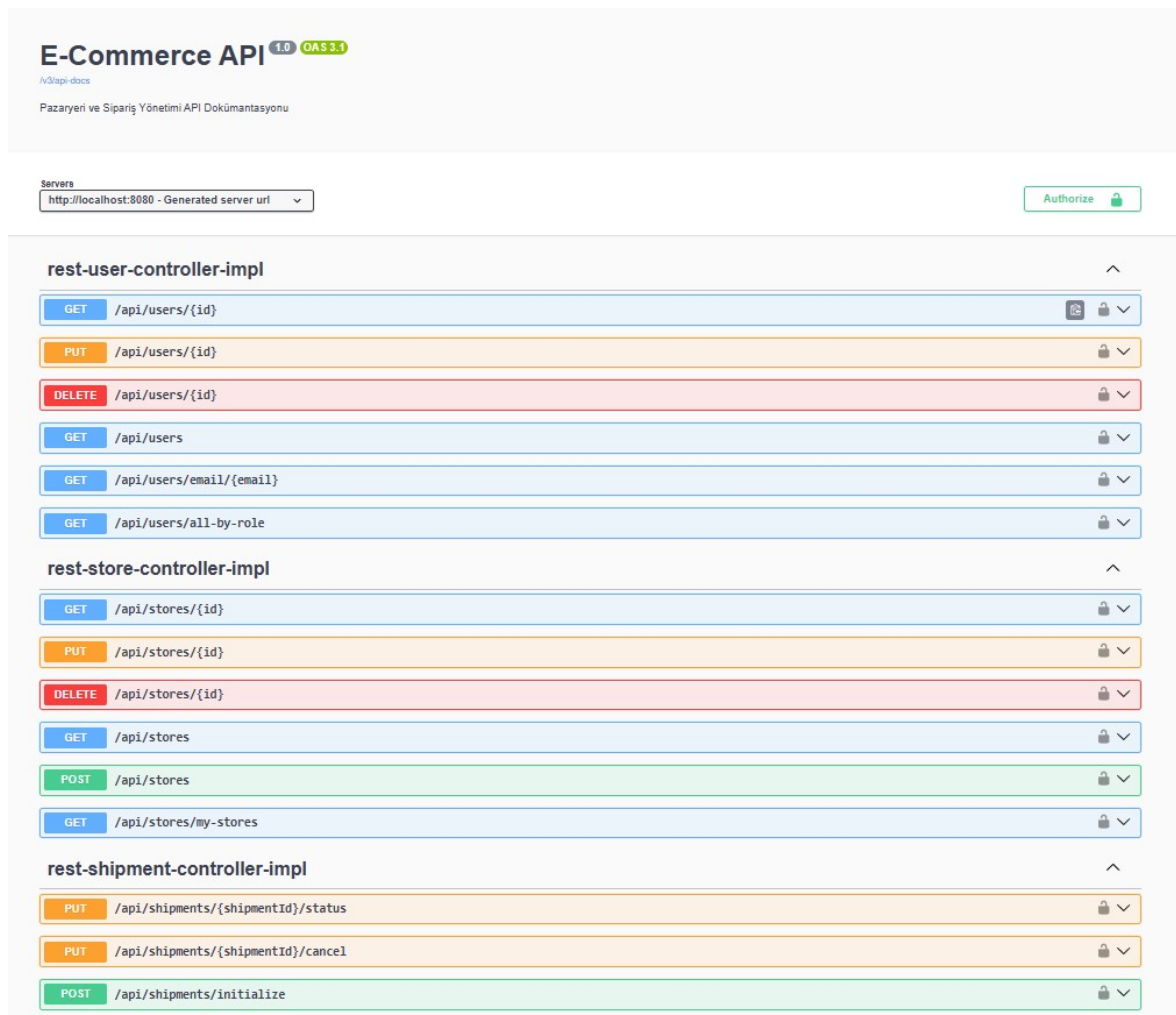


Figure 6: Swagger UI — Full API Endpoint Overview

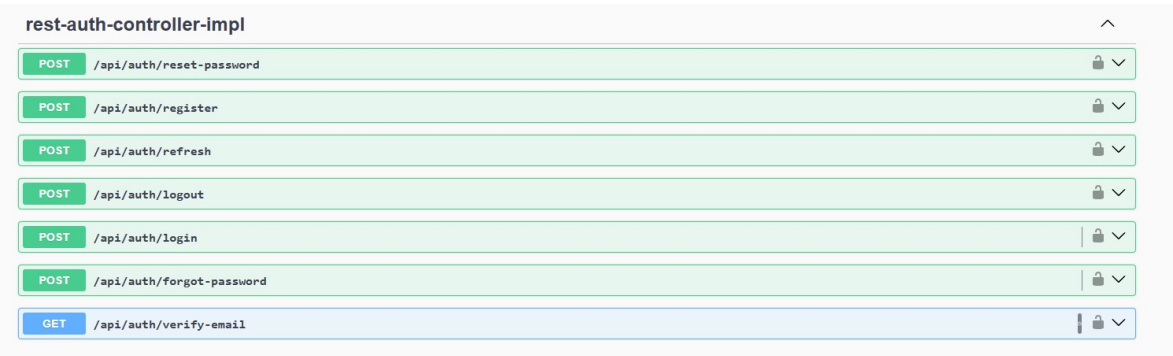


Figure 7: Swagger UI — Auth Controller Detail (Register, Login, Refresh, Logout)



Figure 8: Swagger UI — AI Controller Detail (Role-Based AI Endpoints)

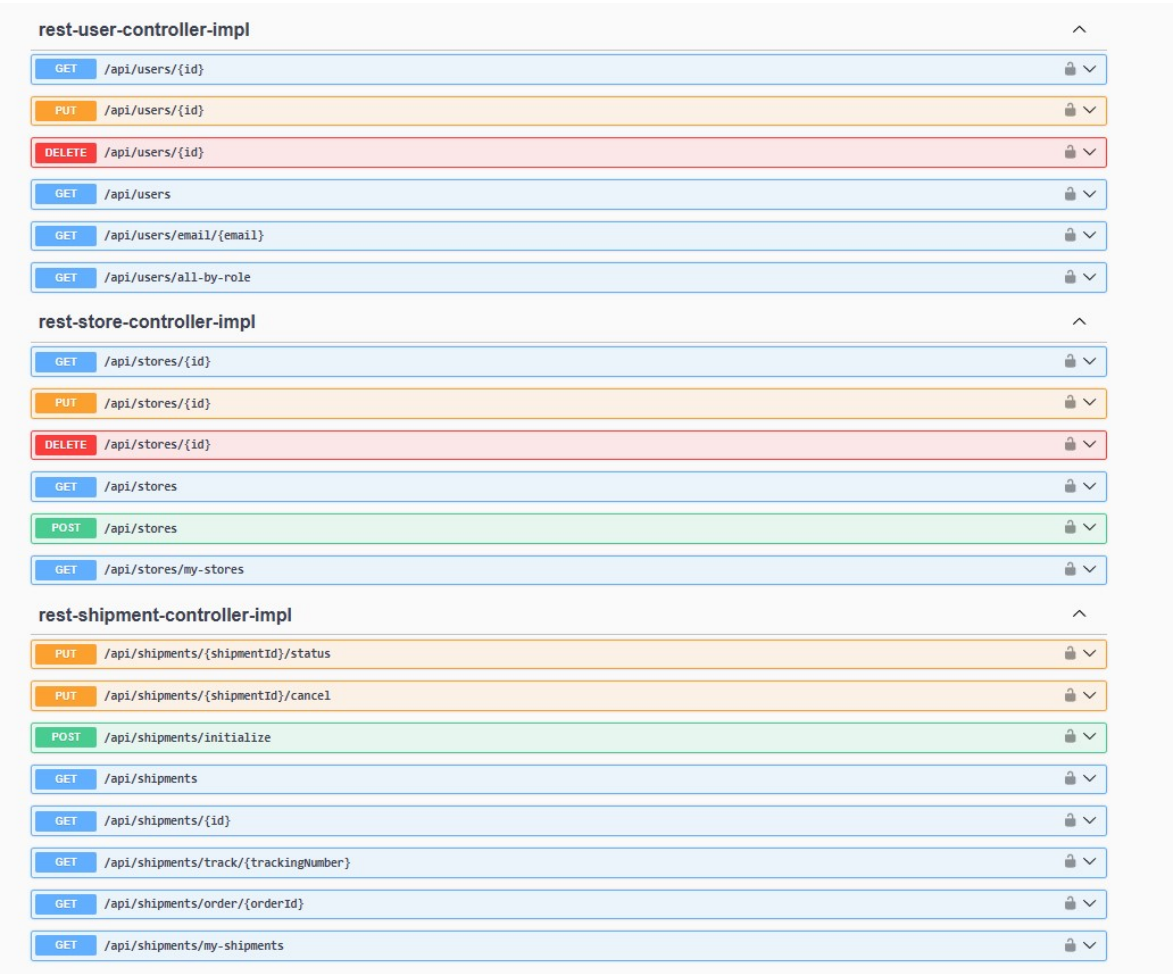


Figure 9: Swagger UI — Endpoint Listing (Users, Stores, Shipments)

rest-review-controller-impl		^
PUT	/api/reviews/{reviewId}	🔒
DELETE	/api/reviews/{reviewId}	🔒
GET	/api/reviews	🔒
POST	/api/reviews	🔒
GET	/api/reviews/{userId}/reviews	🔒
GET	/api/reviews/{id}	🔒
GET	/api/reviews/product/{productId}	🔒
GET	/api/reviews/my-reviews	🔒
GET	/api/reviews/corporate/{storeId}/my-reviews	🔒
DELETE	/api/reviews/my-reviews/{reviewId}	🔒
rest-product-controller-impl		^
GET	/api/products/{id}	🔒
PUT	/api/products/{id}	🔒
DELETE	/api/products/{id}	🔒
GET	/api/products	🔒
POST	/api/products	🔒
GET	/api/products/store/{storeId}	🔒
rest-payment-controller-impl		^
PUT	/api/payments/order/{orderId}/status	🔒
POST	/api/payments/webhook	🔒
POST	/api/payments/refund/order/{orderId}/item/{orderItemId}	🔒
POST	/api/payments/create	🔒
GET	/api/payments/user/{userId}	🔒
GET	/api/payments/order/{orderId}	🔒

Figure 10: Swagger UI — Endpoint Listing (Reviews, Products, Payments)

GET	/api/payments/order/{orderId}/is-successful	⏏
rest-order-controller-impl		⏏
PUT	/api/orders/{orderId}/cancel	⏏
POST	/api/orders/create	⏏
PATCH	/api/orders/sub-order/{suborderId}/status	⏏
GET	/api/orders/{orderId}	⏏
GET	/api/orders/store/{storeId}	⏏
GET	/api/orders/my-orders	⏏
GET	/api/orders/admin/all	⏏
rest-customer-profile-controller-impl		⏏
PUT	/api/customer-profiles/my-profile/upgrade-membership	⏏
PUT	/api/customer-profiles/my-profile/update	⏏
GET	/api/customer-profiles/my-profile	⏏
rest-corporate-update-request-controller-impl		⏏
PUT	/api/corporate-applications/{id}/review	⏏
POST	/api/corporate-applications	⏏
GET	/api/corporate-applications/{id}	⏏
GET	/api/corporate-applications/my-request	⏏
GET	/api/corporate-applications/email	⏏
GET	/api/corporate-applications/all-by-status	⏏
rest-category-controller-impl		⏏
GET	/api/categories/{id}	⏏
PUT	/api/categories/{id}	⏏
DELETE	/api/categories/{id}	⏏
GET	/api/categories	⏏

Figure 11: Swagger UI — Endpoint Listing (Orders, Profiles, Categories)

POST	/api/categories	🔒	▼
rest-cart-controller-impl			
PUT	/api/carts/items/{itemId}/quantity/{quantity}	🔒	▼
POST	/api/carts/add-item	🔒	▼
GET	/api/carts/my-cart	🔒	▼
GET	/api/carts/admin/user/{userId}	🔒	▼
GET	/api/carts/admin/all	🔒	▼
DELETE	/api/carts/items/{itemId}	🔒	▼
DELETE	/api/carts/clear	🔒	▼
DELETE	/api/carts/admin/{cartId}	🔒	▼
rest-address-controller-impl			
PUT	/api/addresses/{addressId}	🔒	▼
DELETE	/api/addresses/{addressId}	🔒	▼
POST	/api/addresses	🔒	▼
GET	/api/addresses/my-addresses	🔒	▼
rest-chat-history-controller-impl			
GET	/api/chat-histories	🔒	▼
POST	/api/chat-histories	🔒	▼
PATCH	/api/chat-histories/{id}/title	🔒	▼
DELETE	/api/chat-histories/{id}	🔒	▼
rest-auth-controller-impl			
POST	/api/auth/reset-password	🔒	▼
POST	/api/auth/register	🔒	▼
POST	/api/auth/refresh	🔒	▼
POST	/api/auth/logout	🔒	▼

Figure 12: Swagger UI — Endpoint Listing (Carts, Addresses, Chat Histories)

POST	/api/auth/login	🔒	▼
POST	/api/auth/forgot-password	🔒	▼
GET	/api/auth/verify-email	🔒	▼
rest-ai-controller-impl			
POST	/api/ai/ask/individual	🔒	▼
POST	/api/ai/ask/corporate	🔒	▼
POST	/api/ai/ask/admin	🔒	▼
rest-analytics-controller-impl			
GET	/api/analytics/store/{storeId}/customers	🔒	▼
rest-audit-log-controller-impl			
GET	/api/admin/audit-logs	🔒	▼

Figure 13: Swagger UI — Endpoint Listing (Analytics, Audit Logs)